

# \_\_\_\_\_ Projet Numérique 2A \_\_\_\_\_



---

Chazalon Hugo / Ginisty Eliot / Debruyne Mathys

Mai 2025

# Table des matières

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                     | <b>1</b>  |
| <b>2</b> | <b>Méthode de résolution numérique</b>                  | <b>2</b>  |
| 2.1      | Equations physique . . . . .                            | 2         |
| 2.2      | Notion de maillage et de champ fluide . . . . .         | 2         |
| 2.2.1    | Maillage spatial . . . . .                              | 3         |
| 2.2.2    | Maillage temporel . . . . .                             | 3         |
| 2.2.3    | Lien entre maillage et précision . . . . .              | 3         |
| 2.2.4    | Intervention du champ fluide . . . . .                  | 3         |
| 2.3      | Discretisation temporelle . . . . .                     | 4         |
| 2.3.1    | Méthode Euler Explicite . . . . .                       | 4         |
| 2.3.2    | Méthode SSP-RK2 pour l'intégration temporelle . . . . . | 5         |
| 2.3.3    | Méthode RK4 pour l'intégration temporelle . . . . .     | 6         |
| 2.4      | Critère de stabilité des schémas . . . . .              | 7         |
| 2.4.1    | Stabilité du schéma Euler explicite . . . . .           | 7         |
| 2.4.2    | Stabilité du schéma RK2-SSP . . . . .                   | 7         |
| 2.4.3    | Stabilité du schéma RK4-SSP . . . . .                   | 8         |
| <b>3</b> | <b>Structure du code et description des fichiers</b>    | <b>10</b> |
| 3.1      | Liste des fichiers nécessaires . . . . .                | 10        |
| 3.2      | Description détaillée des fichiers . . . . .            | 10        |
| 3.3      | Organisation logique du programme . . . . .             | 14        |
| 3.4      | Interpolation bilinéaire . . . . .                      | 14        |
| 3.4.1    | Principe . . . . .                                      | 14        |
| 3.4.2    | Choix des points dans notre cas . . . . .               | 15        |
| <b>4</b> | <b>Validation du code pour des cas simples</b>          | <b>16</b> |
| 4.1      | Chute libre . . . . .                                   | 16        |
| 4.2      | Cas d'une trajectoire ballistique . . . . .             | 16        |
| <b>5</b> | <b>Convergence du pas de temps</b>                      | <b>18</b> |
| <b>6</b> | <b>Convergence en maillage</b>                          | <b>20</b> |
| <b>7</b> | <b>Application du code de calcul</b>                    | <b>21</b> |
| 7.1      | Dispersion de polluant dans l'air . . . . .             | 21        |

# Table des figures

|   |                                                                                    |    |
|---|------------------------------------------------------------------------------------|----|
| 1 | Trajectoires selon différents champs . . . . .                                     | 4  |
| 2 | Trajectoire de Particule pour un schéma instable . . . . .                         | 9  |
| 3 | Interpolation bilinéaire . . . . .                                                 | 14 |
| 4 | Trajectoire pour une chute libre . . . . .                                         | 16 |
| 5 | Trajectoire balistique . . . . .                                                   | 17 |
| 6 | Étude de convergence temporelle pour le suivi lagrangien d'une particule . . . . . | 18 |
| 7 | Étude de convergence en maillage . . . . .                                         | 20 |
| 8 | Champ du vent . . . . .                                                            | 21 |
| 9 | Trajectoire des particules dans le champ de Perlin . . . . .                       | 22 |

# 1 Introduction

## Introduction

L'objectif de ce projet est de développer un code en **Python** permettant de suivre l'évolution d'une particule au cours du temps selon une approche lagrangienne. Cette méthode consiste à se placer dans le référentiel de la particule afin d'analyser sa trajectoire, son état et les forces qui s'exercent sur elle.

La réalisation de ce code a nécessité plusieurs étapes clés, allant de la compréhension du problème physique à la mise en place de schémas numériques adaptés. Chaque étape a été accompagnée de vérifications progressives afin de garantir la cohérence des résultats et la fiabilité des implémentations.

Ce type de code présente un fort potentiel d'application dans de nombreux domaines scientifiques et industriels : mécanique des fluides, transport de polluants, météorologie, combustion, etc. Dans le cadre de ce projet, nous nous intéresserons plus particulièrement à une application simplifiée en combustion, permettant d'illustrer concrètement le fonctionnement et l'intérêt d'une telle modélisation lagrangienne.

## 2 Méthode de résolution numérique

### 2.1 Equations physique

$$m_p \frac{d\vec{u}_p}{dt} = \sum \vec{F}_p \quad (1)$$

L'équation exprime la seconde loi de Newton [5] : le produit de la masse et de l'accélération particulaire est égal à la somme des forces agissant sur cette dernière. Pour une particule immergée dans un écoulement fluide, les forces résultent de l'intégrale de la pression fluide et du tenseur des contraintes visqueuses évaluée à la surface de la particule.

Dans le cas de figure étudié, la majorité des forces s'exerçant sur les particules peut être négligée en vertu des ordres de grandeur en présence [3] :

- On vise des applications où les particules sont soit liquides, soit solides et évoluent dans un fluide gazeux. Ainsi, la densité des particules est très largement supérieure à celle du fluide, typiquement d'un rapport de l'ordre du millier :  $\rho_p \gg \rho_f$ .
- Le diamètre des particules  $d_p$  est compris entre quelques microns et une centaine de microns :  $5 \mu\text{m} < d_p < 100 \mu\text{m}$ . Ainsi, les particules sont supposées petites devant les échelles caractéristiques de l'écoulement.

En utilisant les hypothèses précédentes, on peut montrer que les forces prépondérantes agissant sur la particule sont la force de traînée et la gravité.

$$\sum \vec{F}_p(t) \approx \vec{F}_d(t) + \vec{F}_g = \frac{\pi}{8} \rho_f d_p^2 C_d \|\vec{u}_f - \vec{u}_p\| (\vec{u}_f - \vec{u}_p) + m_p \vec{g} \quad (2)$$

avec  $C_d$  le coefficient de traînée. Ainsi, la vitesse fluide doit être évaluée à la position de la particule. La force de traînée peut également être exprimée comme la différence des vecteurs vitesse fluide et particulaire divisée par un temps de relaxation  $\tau_p$ . En effet, la comparaison des équations (1) et (2) indique que l'ensemble des scalaires précédant le différentiel des vecteurs vitesse fluide et particulaire du terme de traînée dans l'équation (2) est homogène à l'inverse d'un temps. Ce temps caractérise alors la durée requise pour que la particule atteigne la vitesse fluide. Ainsi on obtient :

$$\vec{F}_d(t) = m_p \frac{1}{\tau_p} (\vec{u}_g - \vec{u}_p) \quad (3)$$

avec :

$$\tau_p = \frac{\rho_p d_p^2}{18 \mu_f} \quad (4)$$

Ainsi, l'équation (1) devient

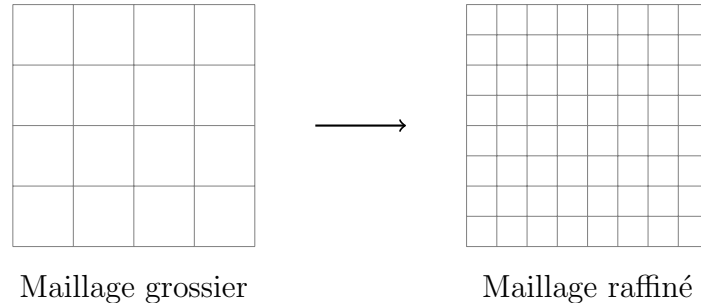
$$m_p \frac{d\vec{u}_p}{dt} = m_p \frac{1}{\tau_p} (\vec{u}_g - \vec{u}_p) + m_p \vec{g} \quad (5)$$

### 2.2 Notion de maillage et de champ fluide

Afin de simuler numériquement le déplacement d'une particule dans un écoulement, il est nécessaire de discrétiser l'espace et le temps. Cette étape fondamentale permet de transformer un problème continu en un problème numérique, exploitable informatiquement.

### 2.2.1 Maillage spatial

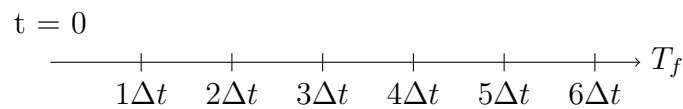
L'espace est divisé en une grille composée de petites cellules carrées (ou volumes de contrôle), constituant le **maillage spatial**. Chaque cellule représente une position potentielle de la particule au cours du temps. Plus le maillage est fin (i.e. plus les cellules sont petites), plus la simulation est capable de capturer les variations locales du champ de vitesse.



### 2.2.2 Maillage temporel

Le temps est également discrétisé en intervalles de durée constante notés  $\Delta t$ , ce qui constitue le **maillage temporel**. La position de la particule est mise à jour à chaque instant  $T_f = N_t \Delta t$ , en fonction du champ de vitesse local.

Un pas de temps trop grand peut faire perdre des informations essentielles, ou rendre la simulation instable. À l'inverse, un pas de temps trop petit augmente fortement le temps de calcul. Il est donc nécessaire de trouver un compromis, souvent contraint par une condition de stabilité, comme la condition CFL (Courant–Friedrichs–Lewy), discutée ultérieurement.



*Découpage temporel uniforme : la position de la particule est actualisée à chaque  $\Delta t$ .*

### 2.2.3 Lien entre maillage et précision

Un maillage fin (spatialement et temporellement) permet une meilleure résolution des phénomènes physiques simulés, au prix d'un coût de calcul plus élevé. Une étude de convergence est donc essentielle pour valider les choix de discrétisation adoptés.

### 2.2.4 Intervention du champ fluide

Une autre composante essentielle du modèle est la prise en compte du **champ de vitesse** ou **champ fluide**, qui impose une contrainte dynamique sur le mouvement des particules.

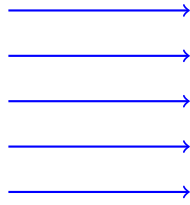
À chaque pas de temps, la particule subit un déplacement induit par le champ de vitesse local à sa position. Ce champ peut varier selon l'espace et/ou le temps, ce qui permet de simuler des écoulements simples ou complexes.

L'intégration du champ fluide rend notre code capable de modéliser des situations réalistes, comme l'injection de carburant dans un moteur ou la dispersion de polluants dans l'atmosphère. Nous avons ainsi implémenté plusieurs types de champs pour tester le comportement des particules dans différents contextes :

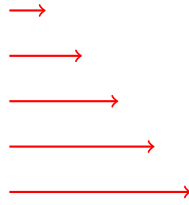
- **Champ uniforme** : le fluide impose une vitesse constante dans tout le domaine ;
- **Champ linéaire** : la vitesse varie linéairement avec la position (ex. : écoulement de cisaillement) ;

- **Champ sinusoïdal** : la vitesse fluctue de manière périodique (ex. : modulation ou oscillation).

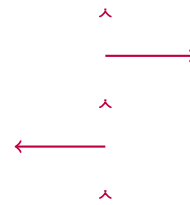
Champ uniforme



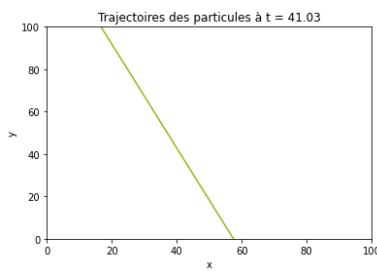
Champ linéaire



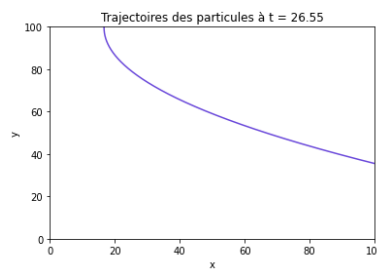
Champ sinusoïdal



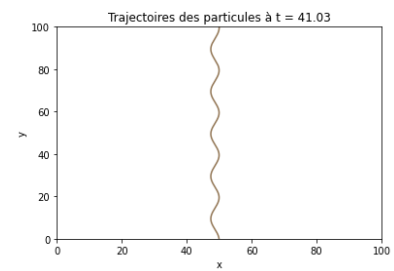
Cette modularité dans la définition du champ fluide est essentielle pour valider le bon fonctionnement de notre code dans des cas tests variés, avant de l'appliquer à des configurations plus complexes.



(a) Trajectoire avec un champ uniforme



(b) Trajectoire avec un champ de cisaillement



(c) Trajectoire pour un champ sinusoïdal

FIGURE 1 – Trajectoires selon différents champs

## 2.3 Discrétisation temporelle

Maintenant que nous avons l'expression (5), il nous faut discrétiser la dérivée temporelle pour la résolution numérique de (5). Cette discrétisation est possible de plusieurs manières mais nous avons choisi les trois suivantes :

- Méthode Euler Explicite
- Méthode Runge-Kutta d'ordre 2 (RK2 - SSP)
- Méthode Runge-Kutta d'ordre 4 (RK4 - SSP)

### 2.3.1 Méthode Euler Explicite

La méthode d'Euler explicite est le schéma numérique le plus simple pour résoudre des **équations différentielles ordinaires** (EDO) de la forme :

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0 \quad (6)$$

La méthode repose sur une approximation de la dérivée par un taux d'accroissement fini.

$$\frac{y_{n+1} - y_n}{\Delta t} \approx f(t_n, y_n) \quad (7)$$

Ce qui donne la formule itérative :

$$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n) \quad (8)$$

où :

- $\Delta t$  est le pas de temps
- $y_n$  est la solution au temps  $t_n$
- $y_{n+1}$  est la solution approchée au temps  $t_{n+1} = t_n + \Delta t$

Dans notre cas, nous avons alors la formule itérative suivante :

$$u_p^{t+1} = u_p^t + \frac{\Delta t}{\tau_p(R_{ep})}(u_g^t - u_p^t) \quad (9)$$

La méthode d'Euler explicite, bien que simple et intuitive, présente des limitations en termes de précision et de stabilité.

### 2.3.2 Méthode SSP-RK2 pour l'intégration temporelle

Nous cherchons à intégrer l'équation du mouvement suivante :

$$m_p \frac{d\vec{u}_p}{dt} = \vec{F}_p$$

où  $m_p$  est la masse d'une particule,  $\vec{u}_p$  sa vitesse, et  $\vec{F}_p$  la force résultante exercée sur la particule. Cette équation peut être reformulée comme une équation différentielle ordinaire (EDO) :

$$\frac{d\vec{u}_p}{dt} = \frac{1}{m_p} \vec{F}_p$$

La méthode **SSP-RK2 (Strong Stability Preserving Runge-Kutta d'ordre 2)** [2] est une méthode explicite à deux étapes qui améliore la stabilité et la précision par rapport au schéma d'Euler explicite. Elle est particulièrement adaptée à des équations présentant des non-linéarités ou des discontinuités, comme celles rencontrées dans les méthodes particulières.

La méthode de Runge-Kutta d'ordre 2 (RK2) permet d'approcher la solution à cette équation en utilisant deux évaluations successives de la dérivée.

Soit  $\Delta t$  le pas de temps. On définit :

$$\begin{aligned} k_1 &= \frac{1}{m_p} F_p(t_n, u_p^n) \\ k_2 &= \frac{1}{m_p} F_p(t_n + \Delta t, u_p^n + h k_1) \\ u_p^{n+1} &= u_p^n + \frac{\Delta t}{2} (k_1 + k_2) \end{aligned}$$

- $k_1$  est la dérivée initiale de la vitesse, calculée à l'instant  $t_n$ ,
- $k_2$  est la dérivée estimée à l'instant  $t_n + \Delta t$ , en supposant que la vitesse a évolué suivant  $\vec{k}_1$ ,
- $u_p^{n+1}$  est la vitesse de la particule au pas de temps suivant.

Cette méthode permet de mieux approximer l'évolution de la vitesse sous l'effet de forces variables, par exemple dans des systèmes où  $\vec{F}_p$  dépend du temps ou de la vitesse (forces non constantes, frottements, champs...).

## Avantages de la méthode SSP-RK2 comparée à Euler explicite

- **Ordre de précision supérieur** : SSP-RK2 est une méthode d'ordre 2 en temps, contre l'ordre 1 pour Euler explicite. Elle permet donc de mieux capturer l'évolution temporelle des phénomènes sans diminuer drastiquement le pas de temps.
- **Préservation de la stabilité non linéaire** : Comme son nom l'indique, la méthode SSP est conçue pour préserver certaines propriétés de stabilité (monotonie, positivité, etc.) qui sont cruciales pour la stabilité des schémas non linéaires (en particulier en présence de chocs ou discontinuités).
- **Meilleure robustesse pour les schémas en volumes finis ou DG** : Contrairement au schéma d'Euler, qui peut générer des oscillations non physiques ou des instabilités dans ces méthodes, SSP-RK2 permet de conserver un comportement physique plus réaliste.
- **Coût modéré** : Bien qu'elle nécessite deux évaluations de la fonction de force  $\vec{F}_p$ , la méthode reste peu coûteuse en comparaison avec des méthodes implicites plus complexes, tout en offrant une précision significativement meilleure qu'Euler.
- **Adaptée aux méthodes à maillage mobile (e.g., particulières)** : L'approche SSP-RK2, grâce à sa formulation explicite et sa stabilité améliorée, est bien adaptée pour les schémas où la géométrie évolue au cours du temps.

En résumé, la méthode SSP-RK2 constitue un bon compromis entre précision, stabilité et simplicité d'implémentation pour l'intégration temporelle dans des simulations numériques non linéaires.

### 2.3.3 Méthode RK4 pour l'intégration temporelle

La méthode **RK4 (Runge-Kutta d'ordre 4)** est une méthode explicite à quatre étapes qui offre une précision très élevée, tout en restant relativement simple à implémenter. Elle est particulièrement utile pour des systèmes dynamiques où la précision temporelle est cruciale.

La méthode s'écrit en quatre étapes :

$$\begin{aligned}
 k_1 &= f(u_p^n) = \frac{1}{m_p} F_p^n \\
 k_2 &= f\left(u_p^n + \frac{\Delta t}{2} k_1\right) = \frac{1}{m_p} F_p^{(1)} \\
 k_3 &= f\left(u_p^n + \frac{\Delta t}{2} k_2\right) = \frac{1}{m_p} F_p^{(2)} \\
 k_4 &= f\left(u_p^n + \Delta t k_3\right) = \frac{1}{m_p} F_p^{(3)} \\
 u_p^{n+1} &= u_p^n + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4)
 \end{aligned}$$

Chaque  $F_p^{(i)}$  est une évaluation de la force à un état intermédiaire, correspondant à la vitesse approximée de la particule.

## Avantages de la méthode RK4 comparée à Euler explicite et SSP-RK2

- **Ordre de précision élevé** : RK4 est une méthode d'ordre 4 en temps, ce qui permet une précision bien supérieure à SSP-RK2 (ordre 2) ou à Euler explicite (ordre 1), pour un même pas de temps.



- **Très bonne précision avec des pas de temps plus larges** : Grâce à son ordre élevé, RK4 peut fournir des résultats précis même avec des pas de temps plus grands, ce qui peut réduire le coût global de la simulation.
- **Fiabilité pour des systèmes dynamiques complexes** : RK4 est souvent utilisée comme référence en raison de sa robustesse et de sa précision, en particulier pour des simulations sensibles à l'erreur temporelle (comme les oscillateurs ou les systèmes chaotiques).
- **Méthode explicite** : Comme SSP-RK2, RK4 reste une méthode explicite, donc facile à implémenter et peu coûteuse par rapport aux schémas implicites qui nécessitent la résolution de systèmes linéaires à chaque pas.
- **Adaptée à des contextes où la précision prime sur la stabilité** : Contrairement à SSP-RK2 qui met l'accent sur la stabilité en présence de discontinuités, RK4 est plus efficace dans des cas où la solution est lisse et où la précision temporelle est prioritaire.

Nous avons implémenté la méthode RK4 [2] dans notre code mais lorsque nous avons tracé la trajectoire de la particule, la différence est de l'ordre de  $10^{-5}$ , donc la précision ne justifie pas le temps de calcul plus long appliqué ici.

## 2.4 Critère de stabilité des schémas

Il est important, lorsqu'on discrétise des équations en utilisant des schémas, de regarder la stabilité de ces derniers. Si l'on se place physiquement dans des conditions où les schémas sont instables, les résultats et notre analyse seront éronnés.

### 2.4.1 Stabilité du schéma Euler explicite

Pour ce faire, nous devons nous rappeler de l'équation (5). Nous nous intéressons seulement à la partie homogène de cette équation, qui devient alors :

$$v_p^{t+1} = v_p^t \left(1 - \frac{\Delta t}{\tau_p}\right) \quad (10)$$

Le schéma est stable lorsque  $|\frac{v_p^{t+1}}{v_p^t}| < 1$ . Ainsi, le critère de stabilité du schéma est alors :

$$\left|1 - \frac{\Delta t}{\tau_p}\right| < 1 \iff \Delta t < 2\tau_p \quad (11)$$

Nous ferons donc attention à nous placer dans cette condition pour avoir le schéma Euler explicite. Cependant, au vu des ordres de grandeurs,  $\tau_p \sim 1s$  et  $\Delta t \sim 0.1s$ , la contrainte apportée n'est pas importante.

### 2.4.2 Stabilité du schéma RK2-SSP

Pour l'équation différentielle :

$$\frac{dy}{dt} = -\frac{1}{\tau_p}y,$$

On applique la méthode de Runge-Kutta d'ordre 2 (RK2) à cette équation.

$$k_1 = \frac{1}{\tau_p}y_n$$

$$k_2 = \frac{1}{\tau_p}y_n \left(1 + \frac{\Delta t}{\tau_p}\right)$$

Le schéma RK2 s'écrit :

$$\begin{aligned} y_{n+1} &= y_n + \frac{\Delta t}{2}(k_1 + k_2) \\ y_{n+1} &= y_n + \frac{\Delta t}{2} \left( \frac{1}{\tau_p} y_n + \frac{1}{\tau_p} y_n \left( 1 + \frac{\Delta t}{\tau_p} \right) \right) \\ &= y_n \left[ 1 + \frac{\Delta t}{2\tau_p} \left( 2 + \frac{\Delta t}{\tau_p} \right) \right] \\ &= y_n \left( 1 + z + \frac{z^2}{2} \right) \end{aligned}$$

où  $z = \frac{\Delta t}{\tau_p}$ .

Le facteur de stabilité du schéma RK2 est donné par :

$$G(z) = 1 + z + \frac{z^2}{2}, \quad \text{avec} \quad z = \frac{\Delta t}{\tau_p}$$

Cette fonction permet d'analyser la stabilité du schéma numérique. Le schéma RK2 est stable lorsque  $|G(z)| \leq 1$ . Nous obtenons alors une stabilité pour

$$\Delta t < 1.7\tau_p$$

Nous ferons donc attention à nous placer dans cette condition pour avoir le schéma RK2-SSP stable.

### 2.4.3 Stabilité du schéma RK4-SSP

Considérons l'équation différentielle test :

$$\frac{dy}{dt} = -\frac{1}{\tau_p}y, \quad y(0) = y_0$$

où  $\frac{1}{\tau_p} \in \mathbb{C}$ . On applique le schéma de Runge-Kutta d'ordre 4 (RK4) avec un pas de temps  $\Delta t$ . On définit  $z = -\frac{\Delta t}{\tau_p}$ .

Le schéma RK4 s'écrit :

$$\begin{aligned} k_1 &= f(t_n, y_n) = \frac{1}{\tau_p} y_n \\ k_2 &= f\left(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} k_1\right) = \frac{1}{\tau_p} \left( y_n + \frac{\Delta t}{2} \frac{1}{\tau_p} y_n \right) = \frac{1}{\tau_p} y_n \left( 1 + \frac{z}{2} \right) \\ k_3 &= f\left(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} k_2\right) = \frac{1}{\tau_p} y_n \left( 1 + \frac{z}{2} + \frac{z^2}{4} \right) \\ k_4 &= f(t_n + \Delta t, y_n + \Delta t k_3) = \frac{1}{\tau_p} y_n \left( 1 + z + \frac{z^2}{2} + \frac{z^3}{4} \right) \end{aligned}$$

Le schéma RK4 donne :

$$y_{n+1} = y_n + \frac{h}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

En factorisant  $y_n$ , on obtient :

$$y_{n+1} = y_n + \frac{\Delta t}{6} \left( \frac{1}{\tau_p} y_n + 2 \frac{1}{\tau_p} y_n \left( 1 + \frac{z}{2} \right) + 2 \frac{1}{\tau_p} y_n \left( 1 + \frac{z}{2} + \frac{z^2}{4} \right) + \frac{1}{\tau_p} y_n \left( 1 + z + \frac{z^2}{2} + \frac{z^3}{4} \right) \right)$$

$$= y_n \left[ 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24} \right]$$

On définit alors la **fonction de stabilité**  $R(z)$  :

$$R(z) = 1 + z + \frac{z^2}{2!} + \frac{z^3}{3!} + \frac{z^4}{4!}$$

$$y_{n+1} = R(z)y_n, \quad \text{avec} \quad z = -\frac{\Delta t}{\tau_p}$$

Ainsi, nous avons la stabilité [4] pour

$$\Delta t < 2.785\tau_p$$

Cependant, voici à quoi ressemble une trajectoire de particule avec un schéma instable.

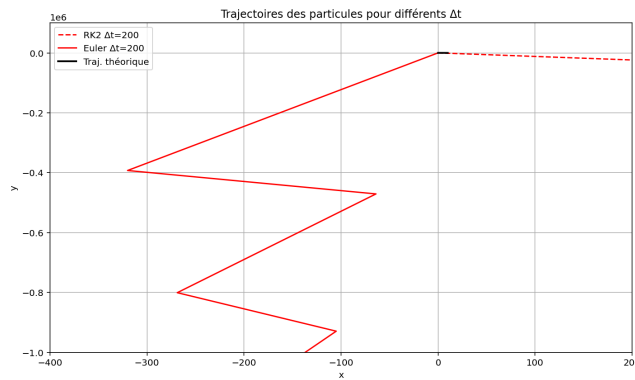


FIGURE 2 – Trajectoire de Particule pour un schéma instable

## 3 Structure du code et description des fichiers

Le projet est organisé de manière modulaire autour de huit fichiers principaux, chacun ayant une fonction spécifique dans la simulation de la dispersion de particules dans un champ de fluide. Cette séparation des responsabilités facilite la lisibilité, la maintenance et l'évolution du code.

### 3.1 Liste des fichiers nécessaires

Tous les fichiers suivants doivent être placés dans le même répertoire pour que le code fonctionne correctement :

- `Code_Final_Main2.py`
- `Solveur.py`
- `Injection_type.py`
- `Initialisation.py`
- `Interpolation.py`
- `Actualisation.py`
- `comparaison.py`
- `data.txt`

### 3.2 Description détaillée des fichiers

#### `Code_Final_Main2.py` – Script principal

Ce fichier constitue le point d'entrée principal de l'application. Il assure la lecture des paramètres de simulation depuis le fichier `data.txt`, l'initialisation des modules nécessaires (champ de fluide, particules, type d'injection, etc.), ainsi que l'exécution de la simulation proprement dite. En fonction du mode sélectionné dans le fichier de configuration, il oriente le programme vers l'un des trois types de calcul implémentés. Il fait appel aux fonctions de calcul définies dans `Solveur.py`, ainsi qu'aux modules d'analyse et de visualisation.

Il convient de distinguer les trois modes de calcul disponibles dans ce projet, chacun correspondant à un objectif spécifique :

1. **Simulation classique** : ce mode permet de simuler le mouvement de plusieurs particules dans un champ de fluide défini, en utilisant l'un des deux schémas numériques disponibles (Euler ou RK2). Il s'agit du mode le plus général, destiné à observer visuellement les trajectoires, sans effectuer d'analyse quantitative particulière. L'utilisateur peut librement choisir le type de champ fluide et la méthode d'injection.
2. **Comparaison des schémas numériques** : dans ce mode, une seule particule est injectée et sa trajectoire est simulée à l'aide des deux schémas d'intégration (Euler et RK2 SSP). Cette simulation est effectuée pour différents pas de temps  $\Delta t$ , ce qui permet d'évaluer l'impact du schéma et du pas de temps sur la précision de la trajectoire obtenue. Ce mode est utilisé pour générer des figures comparatives des erreurs, comme celles présentées dans la suite du rapport.
3. **Étude de convergence spatiale (maillage)** : ce mode vise à analyser la sensibilité des résultats à la résolution spatiale. Une seule particule est simulée à travers plusieurs niveaux de raffinement du maillage. Le champ de fluide et la méthode d'injection sont fixés, de manière à isoler l'effet du maillage sur l'erreur numérique. Ce mode permet de vérifier que la méthode utilisée converge correctement lorsque le maillage est affiné.

Ce découpage en trois modes distincts permet de couvrir à la fois les aspects qualitatifs (visualisation des trajectoires) et quantitatifs (analyse de l'erreur et de la convergence) de la simulation.

### Solveur.py – Moteur de simulation

Contient la fonction `simulate_particles` qui gère l'évolution temporelle des particules au sein du domaine fluide. Cette fonction applique les schémas numériques (Euler explicite ou Runge-Kutta d'ordre 2) pour mettre à jour les positions des particules à chaque pas de temps, en s'appuyant sur le champ de vitesse interpolé localement ainsi que sur les conditions d'injection définies.

Les principes et formulations détaillés de ces schémas numériques sont présentés dans les sections précédentes du rapport, ce qui permet de mieux comprendre leur mise en œuvre dans ce module.

### Injection.type.py – Stratégies d'injection des particules

Ce module définit l'ensemble des stratégies d'injection initiale des particules dans le domaine de simulation. Chaque stratégie correspond à une configuration spatiale et dynamique particulière, permettant d'étudier différents comportements selon le champ de fluide et le schéma numérique utilisés. Le choix de la méthode d'injection est réalisé dans le fichier `data.txt`, en spécifiant l'une des clés suivantes :

"line\_v", "line\_h", "circle", "comparaison", "comparaison\_2"

Les méthodes sont décrites comme suit :

- `line_v` : injecte un ensemble de particules le long d'une ligne verticale placée sur le bord gauche du domaine. Cette configuration permet d'observer la manière dont le fluide entraîne un front de particules réparti uniformément en hauteur.
- `line_h` : injecte les particules sur une ligne horizontale située en bas du domaine. Cette disposition est adaptée pour observer la montée ou la déviation verticale des particules selon les composantes du champ de vitesse.
- `circle` : distribue les particules à l'intérieur d'un cercle centré au point  $(L_x/2, L_y/2)$ , c'est-à-dire au centre du domaine de simulation. Cette configuration permet d'observer la dispersion isotrope (ou anisotrope) des particules autour d'un point donné.
- `comparaison` : injecte une unique particule depuis le coin supérieur droit du domaine, avec une vitesse initiale non nulle. Cette configuration est conçue pour simuler une trajectoire balistique de référence dans un champ fluide nul, et permet la comparaison entre les schémas d'intégration (Euler vs RK2).
- `comparaison_2` : similaire à `comparaison`, mais la particule est lâchée sans vitesse initiale depuis le même coin du domaine. Cette configuration permet d'étudier l'effet du champ fluide seul sur la trajectoire, sans inertie initiale.

Ces stratégies d'injection offrent une grande flexibilité pour simuler des scénarios variés, allant de la dispersion frontale à la trajectoire individuelle de référence, et facilitent ainsi l'étude du comportement des particules dans des champs de fluide complexes.

## Initialisation.py – Génération du champ fluide et des particules

Ce module centralise la définition du champ de vitesse du fluide sur la grille de simulation, ainsi que l'initialisation des particules avec des propriétés physiques potentiellement aléatoires. Il s'inscrit dans une approche lagrangienne, où les particules sont advectées selon les vitesses du fluide interpolées localement. Le module s'appuie sur les paramètres lus depuis un fichier de configuration externe (`data.txt`).

**Champ de vitesse du fluide** Le champ de fluide est stocké dans un tableau à trois dimensions `envt[i, j, :]` représentant les deux composantes de vitesse au centre de chaque cellule de la grille discrète. Plusieurs profils de vitesse peuvent être sélectionnés dynamiquement à partir d'un dictionnaire Python, selon la clé `champ` fournie dans les données.

Les types de champ implémentés sont les suivants :

- `nul` : fluide statique, toutes les composantes de vitesse sont nulles.
- `constant_h` : vitesse constante dans la direction horizontale,  $u = k$ ,  $v = 0$ .
- `constant_v` : vitesse constante dans la direction verticale,  $v = k$ ,  $u = 0$ .
- `lineaire_h` : profil linéaire horizontal,  $u(x) = \alpha x$ ,  $v = 0$ .
- `lineaire_v` : profil linéaire vertical,  $v(y) = \beta y$ ,  $u = 0$ .
- `lineaire_tot` : champ linéaire diagonal, avec  $u(x) = \alpha x$ ,  $v(y) = \beta y$ .
- `cisaillement` : vitesse horizontale décroissante selon  $y$ , modélisant un cisaillement :  $u(y) = \alpha(y_{\max} - y)$ .
- `sinus` : champ oscillant en  $y$ ,  $u(y) = \sin\left(\frac{\pi y}{100}\right)$ .
- `perlin` : champ pseudo-aléatoire bidimensionnel basé sur le bruit de Perlin, utilisé pour simuler une turbulence synthétique.

**Initialisation des particules** La fonction `initialize_particle()` génère les particules une par une, avec les caractéristiques physiques suivantes :

- **Position et vitesse initiales** : déterminées par la stratégie d'injection (`inj[data["inject"]]`).
- **Rayon  $r$  et masse volumique  $\rho_p$**  : peuvent être fixés ou tirés aléatoirement autour de valeurs moyennes (si le mode comparaison est désactivé).
- **Temps caractéristique  $\tau_p$**  :

$$\tau_p = \frac{2r^2\rho_p}{9\mu}$$

- **Masse  $m$**  : calculée à partir du volume sphérique,

$$m = \frac{4}{3}\pi r^3 \rho_p$$

- **Couleur** : générée aléatoirement pour l'affichage.
- **Identifiant unique** : un UUID permet de distinguer chaque particule.

L'ensemble des particules est donc généré avec des propriétés personnalisables, permettant d'étudier leur comportement sous l'action du fluide et de la gravité.

**Lecture des paramètres** Les paramètres nécessaires à la simulation sont lus depuis un fichier texte structuré (`data.txt`). Chaque ligne correspond à une variable :

`L_x, L_y, Nx, Ny, r, rho, nbr_particules,`  
`mu, g, tf, alpha, beta, k,`  
`inject, champ, comparaison, schema, affichage, pas`

Ces valeurs définissent à la fois la géométrie du domaine, les propriétés physiques des particules et du fluide, les conditions d'injection, le champ de vitesse, ainsi que les paramètres numériques de la simulation. Le système de lecture effectue un contrôle du nombre de lignes, et supprime les éventuels commentaires.

### **Interpolation.py – Interpolation du champ**

Effectue l'interpolation bilinéaire du champ de vitesse aux positions des particules. Cette étape est indispensable pour déterminer la vitesse du fluide localement autour de chaque particule, condition nécessaire pour intégrer correctement leur trajectoire. Le fonctionnement détaillé de l'interpolation bilinéaire, ainsi que sa justification mathématique, sont présentés dans la partie 3.4.

### **Actualisation.py – Mise à jour dynamique**

Contient les fonctions de mise à jour de la position et de la vitesse des particules selon les schémas d'intégration :

- `new_vitesse` et `new_position` pour Euler
- `new_vitesse_RK2_SSP2` et `new_position_RK2_SSP2` pour RK2

### **comparaison.py – Analyse comparative et post-traitement**

Ce module regroupe l'ensemble des fonctions dédiées à l'analyse des résultats numériques obtenus dans le cadre de ce projet. Il constitue le cœur du post-traitement utilisé pour valider les méthodes mises en œuvre. Plus précisément, il permet de :

- Comparer visuellement et numériquement les trajectoires calculées avec les schémas d'Euler explicite et de Runge-Kutta d'ordre 2 (RK2)
- Calculer les erreurs entre ces deux méthodes, en particulier l'erreur par rapport à une solution de référence
- Étudier la convergence des schémas numériques en fonction du pas de temps (raffinement du maillage temporel)
- Générer les figures et métriques utilisées dans le rapport pour illustrer la précision, la stabilité et la performance des méthodes implémentées

Ce fichier joue donc un rôle central dans l'évaluation et la validation des résultats présentés dans la suite de ce document.

### **data.txt – Fichier de configuration**

Ce fichier texte contient tous les paramètres nécessaires à la simulation :

- Dimensions du domaine et du maillage
- Nombre de particules
- Type de champ fluide (`champ`)

- Type d'injection (`inject`)
- Type de simulation : 0 (classique), 1 (comparaison), 2 (convergence)
- Schéma numérique utilisé : 0 (Euler), 1 (RK2)

### 3.3 Organisation logique du programme

Le déroulement d'une simulation suit les étapes suivantes :

1. Lecture des paramètres depuis `data.txt`
2. Initialisation du champ fluide selon le choix utilisateur
3. Injection des particules dans le domaine
4. Simulation de leur trajectoire avec le schéma choisi
5. Affichage ou analyse des résultats selon le mode (comparaison ou convergence)

Cette structure permet de facilement modifier, tester ou analyser chaque composant du code indépendamment.

### 3.4 Interpolation bilinéaire

Dans de nombreuses simulations de dynamique des fluides, le champ de vitesse est défini sur une grille cartésienne régulière. Lorsqu'une particule se déplace dans ce champ, sa position  $(x, y)$  ne correspond généralement pas à un nœud de la grille. Il est alors nécessaire d'estimer la valeur du champ (comme la vitesse) à cette position. L'interpolation bilinéaire est une méthode simple, rapide et efficace pour cette estimation.

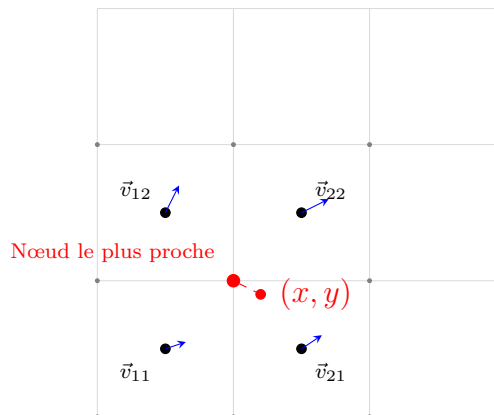


FIGURE 3 – Interpolation bilinéaire

#### 3.4.1 Principe

L'interpolation bilinéaire utilise les valeurs de la fonction en quatre points formant un rectangle pour approximer la valeur en un point intérieur.

Soit une fonction scalaire  $f(x, y)$  définie aux quatre sommets d'un rectangle :

$$\begin{aligned} f_{11} &= f(x_1, y_1), & f_{21} &= f(x_2, y_1), \\ f_{12} &= f(x_1, y_2), & f_{22} &= f(x_2, y_2). \end{aligned}$$

On introduit :

$$dx = x - x_1, \quad dy = y - y_1, \quad \Delta x = x_2 - x_1, \quad \Delta y = y_2 - y_1$$



L'interpolation bilinéaire s'écrit alors :

$$f(x, y) = \frac{dx}{\Delta x} \Delta f_x + \frac{dy}{\Delta y} \Delta f_y + \frac{dx}{\Delta x} \frac{dy}{\Delta y} \Delta f_{xy} + f_{11}$$

où :

$$\Delta f_x = f_{21} - f_{11},$$

$$\Delta f_y = f_{12} - f_{11},$$

$$\Delta f_{xy} = f_{11} + f_{22} - f_{21} - f_{12}.$$

Cette formulation garantit une variation continue et relativement lisse de la fonction interpolée.

### 3.4.2 Choix des points dans notre cas

Dans notre contexte, les quatre nœuds utilisés pour l'interpolation ne sont pas choisis arbitrairement. Pour chaque particule, on identifie d'abord le nœud de grille le plus proche, puis on sélectionne les quatre sommets de la cellule englobante contenant ce nœud. Ces quatre points définissent un rectangle autour de la particule, ce qui garantit que l'interpolation se fait à l'intérieur d'une seule cellule de la grille, assurant ainsi la cohérence locale de l'estimation du champ.

## 4 Validation du code pour des cas simples

Avant de complexifier le code de calcul, nous devons nous assurer que ce que calcul notre code est bien représentatif de la réalité sur des cas simples, où nous pouvons déterminer la solution analytique afin de comparer la véracité de notre solution expérimentale.

Pour cela, nous allons étudier le cas d'une chute libre et le cas d'une trajectoire ballistique. Dans les deux cas, le champs de vitesse du fluide sera nul, pour simplifier au maximum les études.

### 4.1 Chute libre

Le cas d'une chute libre est l'un des cas les plus simples à traiter. Nous considérons une particule sans vitesse initiale. En repartant de l'équation 5, nous pouvons écrire :

$$\frac{d\vec{u}_p}{dt} = -\frac{1}{\tau_p}\vec{u}_p + \vec{g} \quad (12)$$

qui est une équation différentielle du premier ordre linéaire non homogène. La solution s'écrit comme suit :

$$x_p(t) = x_{pi} \quad (13)$$

$$y_p(t) = \tau_p g(t + \tau_p e^{-t/\tau_p}) + x_{piy} \quad (14)$$

La trajectoire de la particule ne dépend que de  $y_p$ , résultat classique avec une trajectoire en chute libre.

En implémentant la solution théorique sur notre code et en simulant notre particule dans ce cas particulier on obtient :

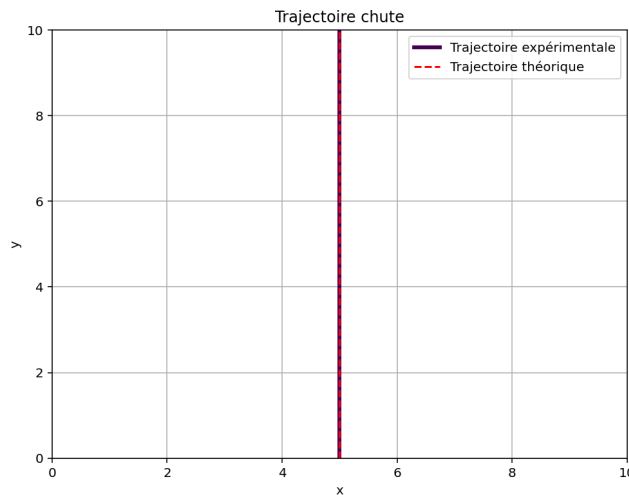


FIGURE 4 – Trajectoire pour une chute libre

On peut observer que les deux trajectoires sont confondues ce qui valide bien le calcul de notre code.

### 4.2 Cas d'une trajectoire ballistique

En repartant de l'équation 5, nous pouvons écrire :

$$\frac{d\vec{u}_p(x_p(t))}{dt} = \frac{-\vec{u}_p(x_p(t))}{\tau_p(Re_p)} + \vec{g} \quad (15)$$

qui est une équation différentielle du premier ordre linéaire non homogène. La solution s'écrit comme suit :

$$y_p(x_p) = \left( \frac{v_{piy} + g\tau_p}{v_{pix}} \right) x_p + g\tau_p^2 \ln \left( 1 - \frac{x_p}{\tau_p v_{pix}} \right) \quad (16)$$

Ainsi, grâce à l'équation (17), nous pouvons tracer sur PYTHON la solution analytique ainsi que la solution donnée par le code.

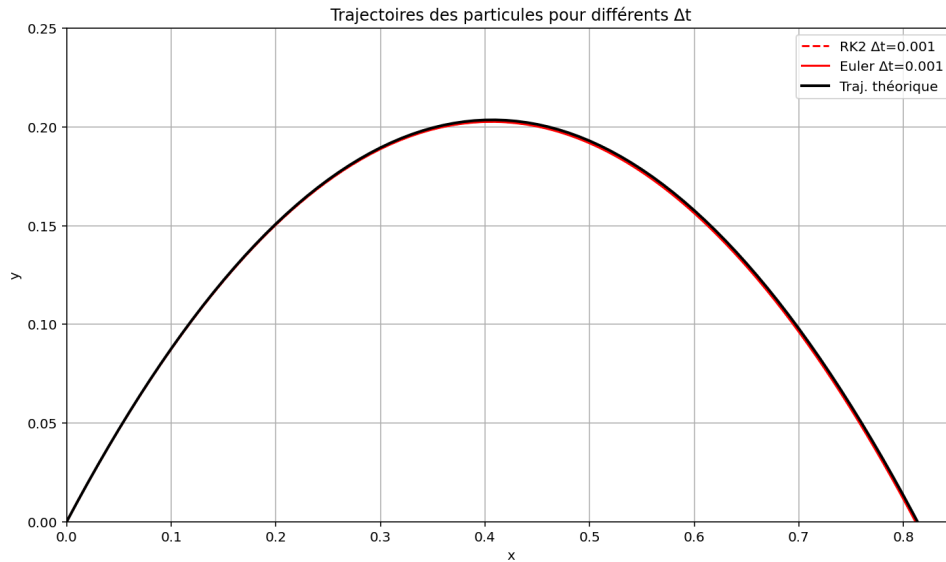


FIGURE 5 – Trajectoire balistique

Sur la figure 5, nous pouvons voir que le code donne un résultat cohérent avec la physique réelle. Nous verrons par la suite, les erreurs apportées par la discrétisation temporelle

## 5 Convergence du pas de temps

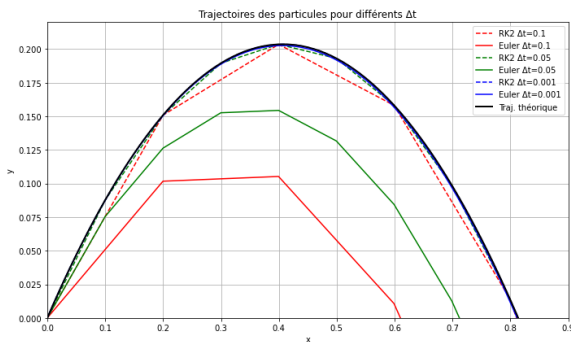
Une simulation numérique consiste à approximer l'évolution d'un phénomène physique en discrétisant le temps en intervalles réguliers appelés pas de temps ( $\Delta t$ ). Le choix de ce paramètre a une influence directe sur la précision et la stabilité des résultats obtenus.

Un pas de temps trop grand peut ne pas capturer correctement certains phénomènes rapides (oscillations, pics, instabilités), ce qui engendre une perte d'information physique. Inversement, un pas de temps trop petit augmente le coût de calcul, parfois inutilement si l'amélioration de précision est négligeable.

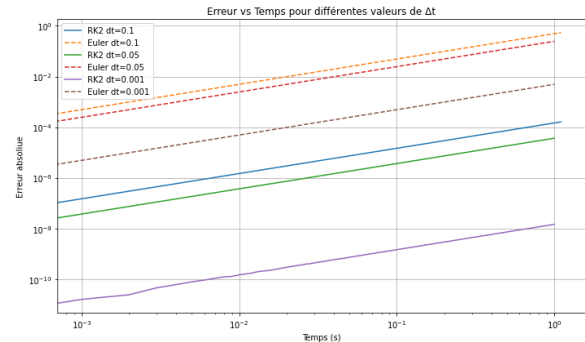
L'étude de convergence temporelle consiste à observer l'évolution de la solution numérique en diminuant progressivement  $\Delta t$ . Si la solution converge vers un comportement stable et cohérent, cela indique que l'erreur de discrétisation temporelle devient négligeable. En revanche, de fortes variations de la solution en fonction de  $\Delta t$  révèlent une erreur importante, suggérant que le pas de temps est trop grand.

Par ailleurs, comme le maillage spatial induit une erreur de discrétisation spatiale, le pas de temps introduit une erreur de discrétisation temporelle. Il est donc essentiel de trouver un bon compromis entre ces deux sources d'erreurs.

Un autre aspect à considérer est l'apparition de **diffusion numérique**, phénomène artificiel lié aux schémas numériques utilisés. Lorsque le pas de temps est trop grand, cette diffusion peut lisser excessivement les gradients de la solution (par exemple, diluer la trajectoire d'une particule), ce qui nuit à la fidélité physique du modèle. Une discrétisation plus fine en temps permet souvent de réduire cet effet parasite.



(a) Comparaison de trajectoires pour différents  $\Delta t$



(b) Évolution de l'erreur relative en fonction de  $\Delta t$

FIGURE 6 – Étude de convergence temporelle pour le suivi lagrangien d'une particule

Les graphiques présentés en figure 6 mettent en évidence l'influence du pas de temps  $\Delta t$  sur la précision numérique. On observe clairement qu'un pas de temps plus petit permet une meilleure approximation de la trajectoire théorique. Plus précisément, le premier graphique (7a) montre que les trajectoires numériques se rapprochent de la courbe théorique à mesure que  $\Delta t$  diminue. Cette tendance est confirmée dans le second graphique (7b), qui illustre la diminution progressive de l'erreur relative en fonction du temps pour différents pas.

Dans le cadre de ce projet, un critère d'erreur maximale a été fixé afin de garantir la qualité des simulations : l'erreur relative à la fin du calcul devait rester inférieure à 0,1. En tenant compte à la fois de ce seuil de tolérance et des contraintes de stabilité évoquées précédemment, un pas de temps compris entre 0,005 et 0,001 s'est révélé être un compromis satisfaisant. Cette plage de valeurs permet d'assurer à la fois la précision des résultats et la stabilité numérique, quel que soit le type de champ de vitesse considéré.

Les figures de la sous-section 6 mettent également en évidence la supériorité de la méthode de Runge-Kutta d'ordre 2 (RK2) par rapport à la méthode d'Euler. Dans le premier gra-

phique (7a), les courbes obtenues avec RK2 — tracées en pointillés — épousent beaucoup plus fidèlement la trajectoire théorique que celles obtenues avec le schéma d'Euler. Cette observation est confirmée par l'analyse quantitative de l'erreur (graphique 7b), qui montre que, pour un même pas de temps, la méthode RK2 est environ **100 fois plus précise** que la méthode d'Euler. Cette amélioration substantielle de la précision justifie pleinement l'utilisation de schémas plus élaborés comme RK2, en particulier lorsque des exigences de fidélité élevée sont imposées.

## 6 Convergence en maillage

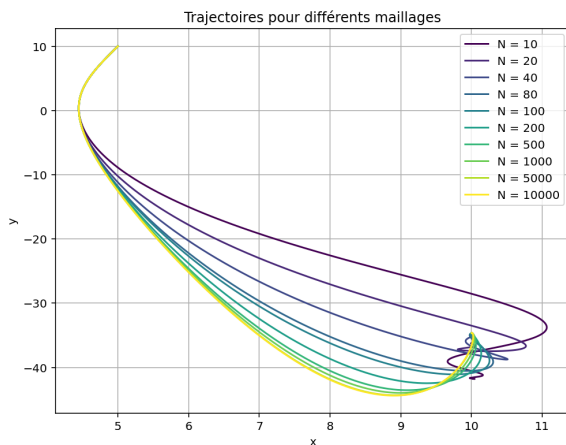
La convergence en maillage est une étape essentielle dans la validation d'une simulation numérique. Elle permet de vérifier que les résultats obtenus sont indépendants de la résolution spatiale choisie. Plus précisément, lorsque le maillage est raffiné (i.e., lorsque la taille des cellules diminue), la solution doit se stabiliser. Si la trajectoire de la particule varie peu entre deux maillages successifs, cela indique que le résultat est fiable et que l'erreur de discrétisation spatiale devient négligeable.

Dans notre cas, cette étude se déroule en plusieurs étapes. Nous faisons tourner la même procédure de calcul pour différentes tailles de maillage et nous extrayons la trajectoire de la particule dans un champ de vitesse donné. Ce champ ne doit être ni trop simple, ni trop complexe :

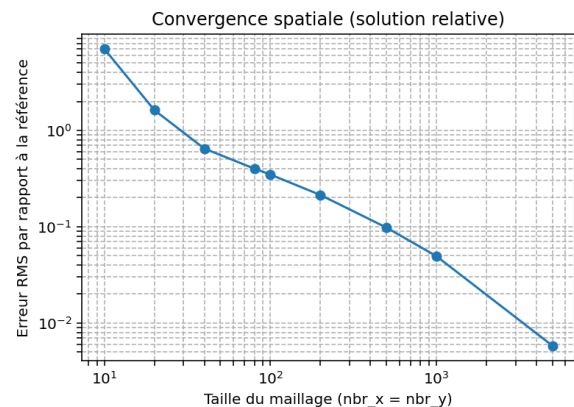
- Un champ de vitesse uniforme ne dépend pas du maillage et ne permet donc pas d'étudier la convergence.
- Un champ trop complexe rend les calculs coûteux sans nécessairement apporter d'informations supplémentaires sur la stabilité numérique.

Une fois les trajectoires obtenues pour chaque maillage, nous comparons chacune d'elles à la trajectoire de référence, calculée avec le maillage le plus fin. L'erreur relative est alors estimée, ce qui nous permet de tracer l'évolution de cette erreur en fonction du nombre de cellules.

Nous pouvons ensuite choisir un maillage adapté, c'est-à-dire suffisamment fin pour garantir une bonne précision, mais sans engendrer un coût de calcul excessif.



(a) Comparaison des trajectoires pour différents maillages



(b) Erreur relative par rapport au maillage de référence

FIGURE 7 – Étude de convergence en maillage

La figure 7 met en évidence la convergence de la trajectoire de la particule. Sur la sous-figure 7a, on observe que les trajectoires tendent vers une solution commune à partir d'un certain niveau de raffinement. Pour les premiers maillages (typiquement autour de  $100 \times 100$  subdivisions), les écarts sont notables, mais ils diminuent rapidement ensuite.

La sous-figure 7b permet de relier directement le niveau de précision souhaité (en termes d'erreur) au maillage nécessaire. Cette approche permet donc de choisir un compromis optimal entre précision et temps de calcul.

## 7 Application du code de calcul

Afin de valider le bon fonctionnement de notre code et d'illustrer son potentiel dans des contextes concrets, nous avons choisi un cas d'application représentatifs de la mécanique des fluides et des problématiques environnementales ou industrielles. Ce cas permet d'exploiter la capacité du code à suivre la trajectoire des particules de manière lagrangienne dans des champs de vitesse non uniformes.

### 7.1 Dispersion de polluant dans l'air

Une des applications possible concerne un problème environnemental majeur : la dispersion de polluants atmosphériques à partir d'une source localisée (cheminée industrielle, accident chimique, etc.). Le suivi de particules dans ce type de situation permet de prévoir la propagation du polluant et d'estimer les zones à risque.

Le champ de vitesse utilisé pour cette simulation est un champ de Perlin [1], un champ synthétique souvent employé pour représenter des écoulements turbulents de manière simplifiée et contrôlée. Il présente des fluctuations spatiales régulières qui imitent certaines propriétés caractéristiques de la turbulence, tout en restant facilement modélisable. Ce type de champ permet de faire apparaître des trajectoires de particules complexes, influencées par des variations locales de la vitesse, sans recourir à la résolution complète des équations de Navier-Stokes. Cela en fait un excellent compromis pour tester la robustesse et la précision de notre algorithme de suivi de particules dans un champ fluide.

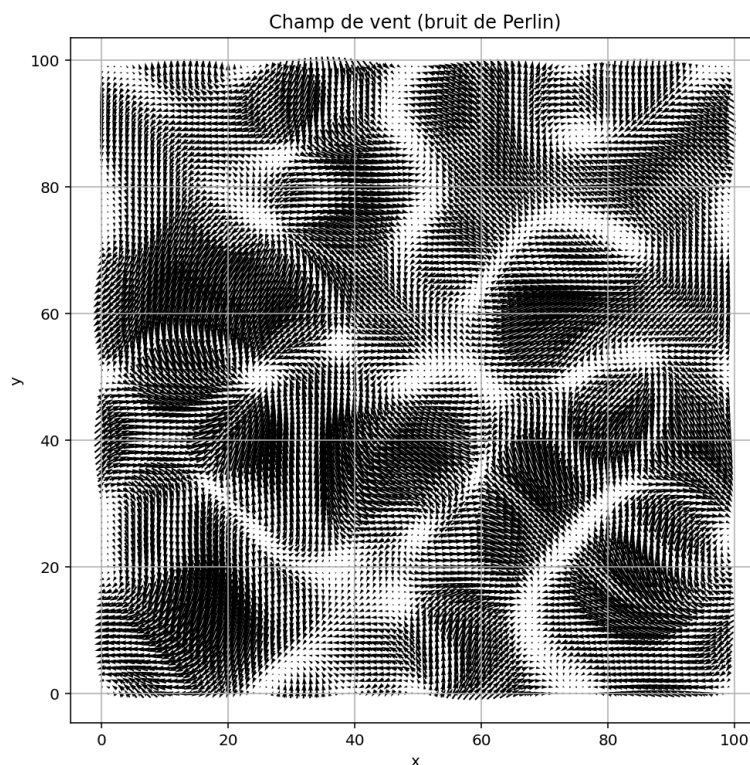


FIGURE 8 – Champ du vent

Ce cas est intéressant car il met en évidence les capacités du code à simuler la trajectoire de particules dans un environnement complexe, où l'évolution temporelle et spatiale du champ de vitesse influence fortement le transport du polluant.

Voici les différentes données que nous avons entré pour simuler notre application :

— Rayon :  $100 \mu m$

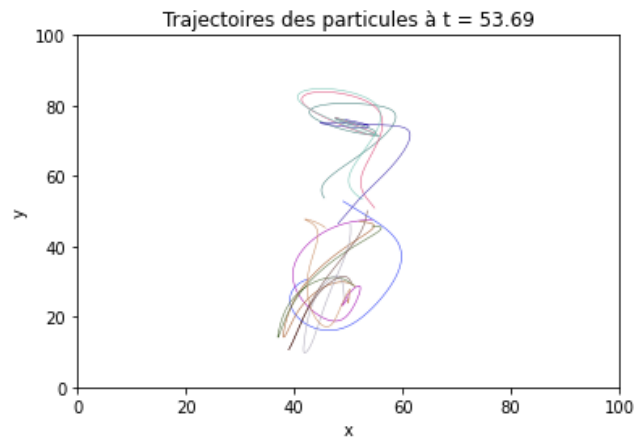


FIGURE 9 – Trajectoire des particules dans le champ de Perlin

- Densité : 1500
- Taille de la zone simulée : 100 m  $\times$  100 m
- Nombre de couches ( niveau de détail ) : 4

Les résultats obtenus permettent de visualiser la dispersion progressive de la particule dans l'espace, avec des trajectoires très différentes selon la position initiale ou les conditions météorologiques. Cette application ouvre des perspectives dans la modélisation des scénarios de pollution et la mise en place de stratégies de prévention.



## Références

---

- [1] Aymeric Augustin. *Flow-noise en temps réel*. PhD thesis, École Polytechnique, 2006.
- [2] femto. Méthode de runge-kutta. *femto-physique*, 2015.
- [3] Nicolas Gindrier, Tuteur Ensimag, and Emmanuel Maitre. Développement d’une méthode de suivi lagrangien de particules dans un code cfd d’ordre élevé pour la les. *mars*, 2018 :31, 2018.
- [4] Claude Metzger. *Méthodes de Runge Kutta de rang supérieur à l’ordre*. PhD thesis, Université Joseph-Fourier-Grenoble I, 1967.
- [5] Isaac Newton. *Philosophiæ naturalis principia mathematica*. *Royal Society*, 1687.